

Unit -4

Hadoop Ecosystem

Hadoop EcoSystem

The Hadoop Ecosystem is a collection of complementary tools and frameworks designed to simplify and optimize the storage, processing, and analysis of large-scale data. While the core Hadoop framework provides HDFS for distributed storage and MapReduce for batch processing, the ecosystem extends these capabilities through specialized tools for data ingestion, processing, storage, and analytics. Together, these components enable organizations to efficiently handle massive datasets, extract insights, and support advanced analytics.

Classification of Ecosystem Tools

Data Ingestion: Tools to bring data into Hadoop from various sources.

- **Flume:** Efficiently collects and transports streaming data such as logs and events into HDFS.
- **Sqoop:** Facilitates bulk transfer of structured data between Hadoop and relational databases.

Storage: Systems for managing and storing large datasets.

- **HDFS (Hadoop Distributed File System):** Provides fault-tolerant, distributed storage for batch processing.
- **HBase:** A NoSQL, column-oriented database that enables real-time read/write access to large datasets.

Data Processing & Analytics: Frameworks for transforming, querying, and analyzing data.

- **Pig:** A high-level scripting platform for batch data processing and transformations.
- **Hive:** Provides an SQL-like interface (HiveQL) for querying and managing large datasets stored in Hadoop.
- **Spark:** A fast, in-memory processing engine capable of both batch and real-time analytics.

Hadoop EcoSystem

Limitations of Core Hadoop

Although Hadoop provides a powerful framework for distributed storage and processing, its core components have certain limitations:

- **Complexity of MapReduce:** Writing raw MapReduce programs requires significant programming effort and deep understanding of distributed processing concepts.
- **Limited Interactivity:** Core Hadoop is primarily batch-oriented and is not suitable for interactive querying or real-time data processing, which makes immediate analysis or rapid decision-making challenging.

How the Hadoop Ecosystem Addresses These Limitations

The Hadoop Ecosystem introduces higher-level tools that overcome these constraints:

- **Pig:** Provides a high-level scripting language to simplify data transformations, reducing the need to write complex MapReduce code.
- **Hive:** Offers SQL-like querying (**HiveQL**), enabling analysts to perform interactive queries on large datasets without programming in Java.
- **Spark:** Supports in-memory processing, allowing fast batch and real-time analytics, making Hadoop suitable for interactive and streaming use cases.
- **HBase:** Provides real-time read/write access to data stored in HDFS, complementing batch storage with low-latency operations.
- **Flume & Sqoop:** Simplify data ingestion from logs and relational databases, reducing the effort required for data integration.

Hadoop EcoSystem (Pig)

1. Introduction to PIG

Apache Pig is a high-level data flow platform developed to simplify the analysis and processing of large datasets in the Hadoop environment. It operates on top of the Hadoop ecosystem and primarily uses Hadoop MapReduce as its execution engine. However, instead of requiring users to write low-level Java MapReduce programs, Pig introduces a simpler, more expressive scripting language known as Pig Latin, making data processing faster, more accessible, and more efficient.

Apache Pig is an advanced data processing tool designed to work with large-scale data stored in the Hadoop Distributed File System (HDFS). It provides:

- A **platform** for creating and executing data analysis workflows.
- A **scripting language** (Pig Latin) that expresses data transformations in a sequence of steps.
- A **higher abstraction layer** above MapReduce, allowing developers to focus on *what* to process rather than *how* to implement the internal logic.

In simple terms, Pig converts Pig Latin scripts into MapReduce jobs automatically, removing the complexity of writing low-level code.

Hadoop EcoSystem (Pig)

Why Pig Was Created

Before Pig, developers had to write **Java-based MapReduce programs** to perform data analysis in Hadoop. This approach had several limitations:

- **High Complexity**
Writing MapReduce programs in Java is tedious and requires deep understanding of distributed programming.
- **Slow Development Cycle**
Even simple data transformations required lengthy code, increasing development time.
- **Lack of Flexibility for Data Analysts**
Analysts or data scientists without strong programming skills found Java MapReduce difficult to use.

To address these challenges, Pig was created to:

- Provide a simple, English-like syntax (Pig Latin).
- Enable rapid development of data flows.
- Hide the internal MapReduce complexities while still benefiting from Hadoop's scalability.

Hadoop EcoSystem (Pig)

Key Features of Pig :

1. Simplicity

Pig Latin allows users to perform operations such as filtering, grouping, sorting, and joining data using simple, readable commands, similar to SQL but more flexible for data flow operations.

Example:

```
A = LOAD 'data.txt' USING PigStorage(',') AS (id:int, name:chararray);  
B = FILTER A BY id > 10;
```

2. Extensibility

Pig allows users to enhance its capabilities by writing their own **User Defined Functions (UDFs)** in languages such as Java, Python, or JavaScript.

This allows Pig to:

- Support custom data transformations
- Integrate domain-specific logic
- Handle complex data types

Key feature of Pig continued...

3. Automatic Optimization

Pig includes an internal optimizer that rearranges the execution steps to improve performance. Users do not need to manually optimize MapReduce tasks; Pig automatically decides the most efficient execution strategy.

This optimization ensures:

- Reduced runtime
- Efficient use of Hadoop cluster resources
- Better scaling for large datasets

4. Ability to Handle Unstructured and Semi-structured Data

Pig is highly suitable for processing:

- Text logs
- JSON-like data
- Clickstream data
- Sensor data
- Nested records

Hadoop EcoSystem (Pig)

Use Cases of Pig :

1. ETL (Extract–Transform–Load)

Pig simplifies the construction of ETL pipelines by allowing easy extraction of raw data, transformation through filtering, cleaning, and formatting, and loading into data warehouses or Hadoop storage. Its scripting nature reduces development time for routine data workflows.

2. Log Processing

Pig is frequently used to process and analyze large volumes of log data such as server logs, clickstream logs, and application activity logs. Its ability to handle semi-structured text makes it useful for monitoring, usage analysis, and detecting system anomalies.

3. Data Cleaning and Preprocessing

Pig offers operators that streamline tasks like removing duplicate records, handling missing or inconsistent values, and standardizing data formats. This makes it highly effective for preparing datasets for further analysis or machine learning.

4. Batch Data Processing

Built on MapReduce, Pig excels at batch-oriented jobs where large datasets are processed periodically. It is used for scheduled data transformations, summarizations, and aggregations that do not require real-time results.

Hadoop EcoSystem (Pig)

Architecture of Apache Pig

Apache Pig is a high-level platform designed to process large datasets on Hadoop. Its architecture is built to simplify data analysis by converting Pig Latin scripts into executable MapReduce jobs, thus hiding the complexity of Hadoop. The main components of Pig architecture are Pig Latin, Pig Runtime, and Grunt Shell.

1. Pig Latin

Pig Latin is a high-level scripting language used to write Pig programs. It allows users to express data transformations such as loading, filtering, grouping, joining, and storing data in a simple, readable format. Pig Latin is designed to handle structured, semi-structured, and unstructured data, making it highly flexible for Big Data processing. Scripts written in Pig Latin are platform-independent, enabling them to run on any Hadoop cluster.

2. Pig Runtime

The Pig Runtime serves as the execution engine of Pig. This component allows users to focus on defining data transformations without worrying about the underlying MapReduce implementation. It is responsible for:

- Translating Pig Latin scripts into a series of MapReduce jobs.
- Optimizing execution automatically for better performance and efficiency.
- Submitting and managing jobs on the Hadoop cluster.

3. Grunt Shell

The Grunt Shell is an interactive command-line interface (CLI) for Pig. Grunt Shell acts as the **user interface**, making Pig more accessible for data analysts and developers. It provides users with the ability to:

- Execute Pig commands interactively.
- Test scripts and debug Pig Latin statements.
- View intermediate results and monitor execution.

Hadoop EcoSystem (Pig)

Execution Modes of Apache Pig

Apache Pig supports multiple execution modes, allowing Pig Latin scripts to run in different environments depending on dataset size, cluster availability, and performance requirements. The main execution modes are Local Mode, MapReduce Mode, Tez Mode, and Spark Mode.

1. Local Mode

Local Mode runs Pig scripts entirely on a single machine using the local filesystem instead of HDFS. It is ideal for development, testing, and debugging with small datasets. Since it executes on a single node, parallel processing is not supported.

Command: `pig -x local`

Use Cases:

- Testing Pig scripts.
- Debugging and learning Pig.
- Validating transformations before cluster deployment.

2. MapReduce Mode (Hadoop Mode)

MapReduce Mode is the default execution mode, in which Pig scripts are converted into MapReduce jobs and executed on a Hadoop cluster. It is suitable for large-scale batch processing of datasets stored in HDFS. Pig handles job creation, execution, and optimization automatically, allowing users to focus on data transformations rather than cluster management.

Command: `pig -x mapreduce`

Use Cases:

- Large-scale batch processing.
- ETL workflows and log analysis.
- Data cleaning and aggregation on HDFS datasets.

Hadoop EcoSystem (Pig)

3. Tez Mode

Tez Mode executes Pig scripts using Apache Tez, which runs workflows as a Directed Acyclic Graph (DAG) rather than a series of MapReduce jobs. This mode provides improved performance and efficiency for complex, multi-stage workflows by reducing job overhead and optimizing resource usage.

Command: `pig -x tez`

Use Cases:

- Complex ETL or batch workflows with multiple joins or groupings.
- Multi-stage data processing pipelines.

4. Spark Mode

Spark Mode runs Pig scripts on Apache Spark, leveraging in-memory processing for faster execution. It is ideal for iterative computations, large datasets, and interactive analysis, such as machine learning pipelines or streaming workflows.

Command: `pig -x spark`

Use Cases:

- Iterative computations (e.g., machine learning pipelines).
- Interactive analysis of large datasets.
- Fast batch processing on Spark clusters.

Hadoop EcoSystem(PIG)

Feature	Apache Pig	Traditional Databases
Data Model	Schema-based, supports nested data types.	Relational model with fixed schema.
Query Language	Pig Latin (a procedural language).	SQL (Structured Query Language).
Data Processing	Batch processing, optimized for large datasets.	Typically supports both batch and real-time.
Scalability	Highly scalable across distributed systems.	Limited by hardware and database architecture.
Performance	Optimized for large-scale data, can utilize MapReduce or Spark.	Performance can vary based on indexing and query optimization.
Flexibility	Schema can evolve over time; supports complex data types.	Schema changes can be complex and require migrations.
Data Storage	Works with HDFS and other big data storage systems.	Generally uses proprietary storage mechanisms.
Use Cases	Best for data transformation, ETL processes, and analytics on large datasets.	Best for transactional processing, real-time queries, and structured data management.
User Base	Typically used by data engineers and analysts in big data contexts.	Used by a wide range of applications, from small to enterprise-level.
Integration	Integrates well with the Hadoop ecosystem.	Integrates with various applications but may require different connectors.

Grunt in Apache Pig

Grunt is the interactive command-line shell of Apache Pig. It provides a user-friendly interface to write, execute, and debug Pig Latin scripts interactively. Through Grunt, users can execute commands line-by-line, inspect data, and monitor the flow of data transformations without creating a full script file.

The main purpose of Grunt is to provide an interactive environment for Pig users. It allows users to:

- **Execute Pig Latin commands immediately** and view results in real-time.
- **Test and debug scripts** incrementally, rather than running the entire Pig script at once.
- **Explore datasets**, inspect schemas, and validate transformations.

Key Features of Grunt

1. Interactive Command Execution

- Users can type Pig Latin commands line-by-line.
- Immediate execution provides quick feedback on data transformations.

2. **Data Loading and Storing** : Load data from files or HDFS into Pig relations, process it, and store the results back into HDFS or local files.

3. **Schema Inspection**: Allows describing the structure of a relation, including field names and data types.

4. **Data Flow Visualization** : Helps understand how data moves and transforms step-by-step within a Pig script.

5. **Quick Debugging and Testing**: Users can run portions of a script to validate logic before executing full workflows.

Grunt in Apache Pig

Common Grunt Commands

Command	Description
dump <relation>	Displays the contents of a relation (results of data transformations).
describe <relation>	Shows the schema (structure) of a relation including field names and types.
illustrate <relation>	Demonstrates the step-by-step data flow for a relation, showing how data is processed through each operator.
quit	Exits the Grunt shell.

Examples:

```
grunt> A = LOAD 'data.txt' USING PigStorage(',') AS (id:int, name:chararray);
grunt> describe A;
grunt> B = FILTER A BY id > 10;
grunt> dump B;
grunt> illustrate B;
grunt> quit;
```

Hadoop EcoSystem(PIG)

Pig Latin

Pig Latin is the high-level scripting language used in Apache Pig for processing large datasets. It provides a simple and procedural way to express data transformations, allowing users to define step-by-step operations without writing complex Java MapReduce programs. Pig Latin scripts are translated into MapReduce, or Spark jobs by the Pig execution engine, making it a powerful tool for both development and production in Hadoop ecosystems.

Key Features of Pig Latin

1. High-Level Language

- Provides a simple syntax to define data transformations.
- Allows users to express complex workflows in a readable and concise way.

2. Procedural Data Flow

- Pig Latin follows a step-by-step execution model, where each statement defines an operation on data.
- Users can load data, apply transformations, filter, group, join, and store results sequentially.

3. Supports Nested Data Types

- Pig Latin can handle complex data structures such as tuples, Social media post, order details enabling sophisticated processing of semi-structured and unstructured data.

4. Extensible via UDFs (User-Defined Functions)

- Users can write their own functions in Java, Python, or other supported languages to extend Pig's capabilities.

Pig Latin continued...

syntax:

Here are some common Pig Latin operations:

- **Loading Data:** `data = LOAD 'input.txt' USING PigStorage(',') AS (field1:chararray, field2:int);`
- **Filtering Data:** `filtered_data = FILTER data BY field2 > 100;`
- **Grouping Data:** `grouped_data = GROUP filtered_data BY field1;`
- **Aggregating Data:** `result = FOREACH grouped_data GENERATE group, COUNT(filtered_data);`
- **Storing Data:** `STORE result INTO 'output' USING PigStorage(',');`

Example Pig Latin Script

Here's a simple example that demonstrates loading, filtering, and storing data:

```
-- Load data from a text file
data = LOAD 'input.txt' USING PigStorage(',') AS (name:chararray, age:int);

-- Filter for people older than 30
filtered_data = FILTER data BY age > 30;

-- Store the result
STORE filtered_data INTO 'output' USING PigStorage(',');
```


User Defined functions (pig)

User-Defined Functions (UDFs) are custom functions created by users to extend the capabilities of Pig Latin. While Pig provides a rich set of built-in operators and functions for data transformation, there are situations where specialized processing or computations are required. UDFs allow users to implement custom logic to meet these requirements.

The main purposes of UDFs in Pig are:

- To perform custom transformations that are not supported by built-in Pig operators.
- To implement complex business logic on datasets.
- To reuse and modularize Pig scripts for better readability and maintainability.

For example, you might create a UDF to calculate a custom score, format strings, or perform complex aggregations.

Steps to Use a UDF in Pig

1. Create the UDF

- Implement the custom logic in Java (or supported language).
- Extend Pig's EvalFunc class for evaluation functions.

User Defined functions (pig)

2. Register the UDF

- Make the UDF available to Pig using the REGISTER command.

```
REGISTER 'myudf.jar';
```

3. Use the UDF in Pig Latin

- Call the function in Pig Latin scripts just like built-in functions.

Example: Simple Java UDF:

Suppose we want a UDF to convert a string to uppercase:

1. Java Code (MyUpper.java):

```
import org.apache.pig.EvalFunc;
import org.apache.pig.data.Tuple;
import java.io.IOException;

public class MyUpper extends EvalFunc<String> {
    public String exec(Tuple input) throws IOException {
        if (input == null || input.size() == 0)
            return null;
        String str = (String) input.get(0);
        return str.toUpperCase();
    }
}
```

2. Using in Pig Latin

```
REGISTER 'myupper.jar';
DEFINE MyUpper com.example.MyUpper();
A = LOAD 'students.txt' USING PigStorage(',') AS (id:int, name:chararray, marks:int);
B = FOREACH A GENERATE MyUpper(name), marks;
DUMP B;
```

Data Processing operators (Pig)

Apache Pig provides a rich set of data processing operators that allow users to manipulate, transform, and analyze large datasets. These operators form the core of Pig Latin scripts and make it possible to express complex data workflows in a simple, high-level manner. Here's an overview of the key operators:

1. LOAD

Description: Loads data from a specified location into a Pig relation.

Example

```
data = LOAD 'input.txt' USING PigStorage(',') AS (field1:chararray, field2:int);
```

2. FILTER

Description: Filters the data based on a specified condition.

Example:

```
filtered_data = FILTER data BY field2 > 100;
```

Unit:4 Hadoop EcoSystem(PIG)

3. FOREACH

Transforms each element in a relation.

```
transformed_data = FOREACH filtered_data GENERATE field1, field2 * 2;
```

4. GROUP

Groups data by a specific field.

```
grouped_data = GROUP data BY field1;
```

5. JOIN

Joins two relations based on a common key.

```
joined_data = JOIN data1 BY field1, data2 BY field1;
```

Unit:4 Hadoop EcoSystem(PIG)

6. CROSS

Creates the Cartesian product of two relations.

```
crossed_data = CROSS data1, data2;
```

7. ORDER BY

Orders the data based on specified fields.

```
ordered_data = ORDER data BY field1 DESC;
```

8. LIMIT

Limits the number of tuples in the output.

```
limited_data = LIMIT ordered_data 10;
```

Unit:4 Hadoop EcoSystem(PIG)

9. STORE

Stores the result back into the file system.

```
STORE transformed_data INTO 'output' USING PigStorage(',');
```

10. DISTINCT

Removes duplicate tuples from a relation.

```
distinct_data = DISTINCT data;
```

Hadoop EcoSystem(HIVE)

Apache Hive is a high-level data warehousing and querying framework developed to simplify the analysis of large datasets within the Hadoop ecosystem. It operates on top of Hadoop and uses distributed processing engines such as MapReduce or Spark to execute queries. Instead of requiring users to write complex MapReduce programs, Hive provides an SQL-like query language called HiveQL, which makes data analysis more intuitive, efficient, and accessible especially for users familiar with traditional relational databases

Primarily designed for batch-oriented analytics, Hive is widely used for data summarization, reporting, and large-scale analysis, making Hadoop accessible to users familiar with traditional relational databases.

Data Storage in Hive/Tables in Hive

In Hive, data is stored in table-like structures, optimized for analytical processing. Key storage features include:

- **Tables:** Data is organized into tables, which can be managed (internal) or external.
 - **Managed Tables:** Hive manages both the data and metadata. Dropping the table deletes the data.
 - **External Tables:** Hive manages only the metadata; the data remains in HDFS even if the table is dropped.
- **Partitions:** Tables can be divided into partitions based on a column (e.g., date or region) to improve query performance.
- **Buckets:** Data within partitions can be further divided into buckets (or clusters) for efficient sampling and joins.
- **Schema-on-Read:** Hive applies the schema when reading the data rather than writing it, allowing flexible handling of structured and semi-structured datasets.

Key Features of Hive

1. **Declarative SQL-Like Query Language (HiveQL):** Provides a high-level, declarative query language similar to SQL, enabling users to perform complex data analysis without writing low-level MapReduce programs.
2. **Data Warehousing Support:** Facilitates structured data organization through databases, tables, partitions, and buckets, supporting both structured and semi-structured datasets.
3. **Scalable Distributed Processing:** Converts queries into distributed execution plans on MapReduce or Spark, leveraging Hadoop's scalability, parallelism, and fault-tolerance.
4. **Batch-Oriented Analytical Processing:** Optimized for large-scale batch analytics, summarization, and reporting, rather than real-time transactional processing.
5. **Support for Multiple Storage Formats:** Compatible with Text, ORC, Avro, and other file formats, enabling efficient storage, compression, and performance optimization.
6. **Extensibility Through User-Defined Functions:** Supports UDFs, allowing users to implement custom analytical logic beyond the built-in functionality.
7. **Seamless Integration with the Hadoop Ecosystem:** Interoperates with HDFS, YARN, Pig, Spark, and HBase, supporting comprehensive big data workflows.
8. **Centralized Metadata Management (Metastore):** Maintains metadata for tables, schemas, columns, partitions, and storage formats, which enhances query planning, execution efficiency, and data consistency.

Hadoop Ecosystem(Hive)

Hive Shell

The Hive Shell is a command-line interface (CLI) that enables users to interact with Apache Hive. It provides a platform to execute HiveQL queries, manage databases and tables, and access metadata. Essentially, it acts as the primary interface between the user and Hive, allowing command execution without directly handling Hadoop's low-level components like MapReduce.

Key Features of Hive Shell

1. Interactive Query Execution

- Users can execute HiveQL queries interactively and see results immediately.
- Supports batch execution of scripts containing multiple Hive commands.

2. Query Validation

- Hive Shell performs syntax checking and semantic validation of queries before execution.
- Ensures that tables, columns, and databases referenced in queries exist in the Hive Metastore.

3. Access to Metadata

- Provides commands to retrieve information about databases, tables, columns, and partitions.
- Facilitates understanding of the structure of stored data before performing operations.

4. Query Results Retrieval

- Displays the output of queries directly on the CLI.
- Enables redirection of query results to local files for further processing if required.

Common Hive Shell Commands

1. **show databases;**

Lists all databases available in Hive.

Example:

```
hive> show databases;
```

2. **use db_name;**

Switches the current database context to a specified database.

Example:

```
hive> use sales_db;
```

3. **show tables;**

Lists all tables in the current database.

Example:

```
hive> show tables;
```

Output:

```
customers  
orders  
products
```

Common Hive Shell Commands

4. **describe table_name;**

Displays detailed information about a table's structure, including columns, data types.

Example:

```
hive> describe customers;
```

output:

```
customer_id  int
name         string
email        string
```

5. **quit;**

Exits the Hive Shell.

Example:

```
hive> quit;
```

Hadoop EcoSystem(HIVE)

Hive Services

Apache Hive provides several services to facilitate data warehousing and querying in a Hadoop ecosystem. Here are the key services associated with Hive:

1. Hive Metastore

Purpose: Stores metadata about Hive tables, partitions, and databases.

Functionality: Manages schema information and allows for efficient data retrieval. It can be accessed using a relational database like MySQL or PostgreSQL.

2. HiveServer2

Purpose: Acts as a server for clients connecting to Hive.

Functionality: Provides a interface for applications to interact with Hive, supporting multiple client connections and security features. It allows for better concurrency and performance.

Hadoop EcoSystem(HIVE)

3. Hive CLI (Command Line Interface)

Purpose: Provides an interactive shell for running HiveQL queries.

Functionality: Users can execute commands directly, manage databases and tables, and run scripts.

4. Hive Web Interface (Hive UI)

Purpose: A web-based graphical interface for Hive.

Functionality: Allows users to run queries and manage their Hive environment without needing to use command-line tools.

Hadoop EcoSystem(HIVE)

Hive Metastore

The **Hive Metastore** is a central and critical component of Apache Hive that stores metadata about the data managed by Hive. Metadata refers to the structural information of datasets, such as the schema, table definitions, column types, partitions, and storage locations. By maintaining this metadata separately from the actual data, Hive enables efficient query planning, execution, and interoperability with other Hadoop ecosystem tools.

Key Functions of Hive Metastore

1. Metadata Storage:

- Stores information about databases, tables, columns, partitions, and data types.
- Tracks table properties, storage formats (ORC, Text, Avro), and file locations in HDFS.
- Maintains details of indexes and constraints if used.

2. Schema Management:

- Enables schema-on-read, meaning the schema is applied when the data is queried rather than at the time of data ingestion.
- Allows Hive to work with both structured and semi-structured datasets.

Hadoop EcoSystem(HIVE)

3. Partition and Bucket Metadata:

- Stores information about table partitions (e.g., date, region) to optimize query execution.
- Maintains bucket information, enabling efficient joins and sampling.

4. Query Optimization Support:

- Provides metadata required by the Hive query compiler to generate an efficient execution plan.
- Supports predicate pushdown, column pruning, and partition pruning to reduce the amount of data scanned during queries.

5. Interoperability:

- Acts as a centralized metadata service for multiple Hive clients or other Hadoop ecosystem tools such as Spark, Pig, and ETL frameworks.
- Allows multiple engines to access consistent metadata without duplicating information.

6. Storage:

- The Metastore itself is typically implemented as a relational database (commonly MySQL, PostgreSQL) separate from the actual dataset in HDFS.
- Hive queries interact with the Metastore to retrieve metadata, but the data remains in HDFS or other storage layers.

Hadoop EcoSystem(HIVE)

Types of Metastore

- **Embedded Metastore:**
 - Runs in the same process as the Hive service.
 - Suitable for single-user setups or development/testing environments.
- **Remote/Standalone Metastore:**
 - Runs as a separate server process, accessible over the network.
 - Suitable for multi-user or production environments, allowing multiple Hive clients to access the same metadata simultaneously.

Importance of Hive Metastore

1. **Centralized Metadata Management:** Provides a single source of truth for all Hive tables and partitions.
2. **Query Efficiency:** Enables faster query compilation and execution by providing schema and partition information.
3. **Data Governance:** Supports consistency, auditability, and access control over large datasets.
4. **Ecosystem Integration:** Allows Hive to integrate seamlessly with other Hadoop tools that require metadata.

Hadoop EcoSystem(HIVE)

Comparison between Apache Hive and traditional relational databases:

Feature	Apache Hive	Traditional Databases
Data Model	Schema on read (flexible, can handle unstructured data)	Schema on write (fixed schema)
Query Language	HiveQL (similar to SQL)	SQL
Storage	Uses HDFS for storage of large datasets	Uses traditional file systems or storage engines
Performance	Optimized for batch processing; slower for real-time queries	Optimized for transactional and real-time queries
Scalability	Highly scalable (designed for big data)	Limited scalability; vertical scaling
Concurrency	Limited concurrency; designed for batch jobs	High concurrency support with ACID transactions
Data Types	Supports various data formats (JSON, Avro, Parquet, etc.)	Fixed data types defined in schema
Partitioning	Supports partitioning for optimized queries	Usually supports partitioning but with different mechanisms
Deployment	Runs on Hadoop ecosystem; requires setup of Hadoop cluster	Standalone installations (e.g., MySQL, PostgreSQL)
Use Cases	Data warehousing, analytics, ETL processes	Transactional applications, operational workloads

Hadoop EcoSystem(HIVE)

HiveQL (Hive Query Language)

HiveQL (Hive Query Language) is the primary language used in Apache Hive to define, query, and manage data stored in Hive tables. It is a high-level, SQL-like declarative language that abstracts the complexity of writing low-level MapReduce, or Spark jobs. HiveQL allows users with a background in relational databases to interact with big data stored in Hadoop using familiar SQL constructs, while automatically converting queries into distributed execution plans.

Key Features of HiveQL

1. SQL-Like Syntax:

- HiveQL closely resembles SQL standard, with support for SELECT, INSERT, UPDATE, DELETE, JOIN, GROUP BY, ORDER BY, and aggregation functions.
- This similarity makes it accessible to users familiar with relational database management systems (RDBMS).

2. Data Definition Language (DDL):

HiveQL provides commands to define and manage table structures:

- **CREATE DATABASE / DROP DATABASE** – Create or delete databases.
- **CREATE TABLE / DROP TABLE** – Create or delete tables in Hive.
- **ALTER TABLE** – Modify table schema or partitions.
- **CREATE VIEW / DROP VIEW** – Define virtual tables based on queries.

Key Features of HiveQL

3. Data Manipulation Language (DML):

HiveQL supports operations for manipulating and querying data:

- **SELECT** – Retrieve data with filtering, sorting, and aggregation.
- **INSERT INTO / OVERWRITE** – Load data into tables or partitions.
- **LOAD DATA** – Import data from HDFS or local file systems.
- **INSERT OVERWRITE DIRECTORY** – Export query results to HDFS directories.

4. Partition and Bucket Awareness:

- HiveQL allows queries to **filter data using partitions**, reducing the amount of data scanned and improving query performance.
- Supports operations on **bucketed tables**, enabling efficient joins and sampling.

5. Functions and Expressions:

- Built-in **aggregate functions**: COUNT(), SUM(), AVG(), MAX(), MIN().
- Built-in **string, date, and mathematical functions** for data transformation.
- Supports **user-defined functions** for custom processing.

6. Joins and Subqueries:

- Supports **INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN, CROSS JOIN**, allowing complex data analysis across multiple tables.
- Supports **nested subqueries** and common table expressions (CTEs) for modular query writing.

Key Features of HiveQL

7. External Table Support:

- HiveQL allows querying **external tables**, enabling analysis of data stored outside Hive without importing it.

8. Extensibility and Integration:

- HiveQL integrates with other Hadoop tools, such as Spark and Pig, and allows defining custom functions to extend its functionality.

9. Batch-Oriented Execution:

- HiveQL queries are designed for large-scale, batch-processing workloads. Real-time or transactional updates are limited compared to traditional RDBMS.

Hadoop EcoSystem(HIVE)

HiveQL(Basic Syntax)

1. Creating Databases

```
CREATE DATABASE my_database;
```

2. Using a Database

```
USE my_database;
```

3. Creating Tables

```
CREATE TABLE employees (  
    id INT,  
    name STRING,  
    salary FLOAT  
) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
```

4. Loading Data

```
LOAD DATA INPATH '/path/to/data.csv' INTO TABLE employees;
```

5. Querying Data

1. Select All:

```
SELECT * FROM employees;
```

2. Filtering Data:

```
SELECT name, salary FROM employees WHERE salary > 50000;
```

Hadoop EcoSystem(HIVE)

HiveQL(Basic Syntax)

6. Aggregate Functions

```
SELECT COUNT(*), AVG(salary) FROM employees GROUP BY department;
```

7. Joining Tables

```
SELECT a.name, b.department  
FROM employees a  
JOIN departments b ON a.dept_id = b.id;
```

Hadoop EcoSystem(HIVE)

User-Defined Functions (UDFs) in Hive

In Apache Hive, a User-Defined Function (UDF) is a custom function that extends the built-in functionality of HiveQL. UDFs allow users to perform operations that are not natively supported, such as complex transformations, calculations, or string manipulations, directly within Hive queries. They provide flexibility for specialized analytical or data-processing tasks.

Types of Hive UDFs:

- **UDF (Scalar Function):** Operates on a single row and returns a single value.
Example: `my_custom_upper(name)` converts a string column to uppercase.
- **UDAF (User-Defined Aggregate Function):** Operates on multiple rows to produce a single aggregated result.
Example: `my_custom_avg(salary)` calculates a customized average.
- **UDTF (User-Defined Table-Generating Function):** Returns multiple rows for each input row.
Example: `explode(array_column)` expands an array into multiple rows.

Implementation:

- Typically implemented in Java and packaged as a JAR file.
- Can be added to Hive using the ADD JAR command and registered using CREATE FUNCTION

Example:

```
ADD JAR /path/to/my_udf.jar;  
CREATE FUNCTION my_upper AS 'com.example.hive.MyUpperUDF';  
SELECT my_upper(name) FROM employees;
```

Hadoop EcoSystem(HBASE)

HBase is a distributed, scalable NoSQL database that runs on top of the Hadoop Distributed File System (HDFS). It is inspired by Google's Bigtable and provides an efficient way to store and process extremely large volumes of data in a fault-tolerant, horizontally scalable, and real-time manner. As a major component of the Hadoop ecosystem, it supports fast, random read/write operations across massive datasets.

Unlike traditional relational databases (RDBMS) that store data in rows and enforce rigid schemas, HBase is a column-oriented, schema-less database designed for high throughput and low latency operations. Built closely with HDFS, it ensures reliability and seamless integration with other Hadoop components such as MapReduce, enabling real-time access and processing of big data.

Characteristics of HBase

1. Column-Oriented Storage Model

HBase organizes data into tables, which are further divided into column families, columns (qualifiers), and cells. Each cell is uniquely identified by a row key, column family, column qualifier, and timestamp. This column-oriented structure enables efficient handling of sparse datasets and supports fast access to specific columns.

2. NoSQL and Key-Value Data Model

HBase follows a flexible NoSQL model where each row is identified by a unique row key, and values are stored in individual cells. It supports multiple versions of data through timestamps and does not require a fixed schema, allowing columns to be added dynamically within defined column families.

Characteristics of HBase

3. **Real-Time Random Read/Write Access**

HBase provides low-latency, millisecond-level random read and write operations. Unlike HDFS, which supports only batch processing, HBase enables real-time access to large datasets, making it suitable for operational workloads.

4. **Horizontal Scalability**

HBase scales efficiently across hundreds or thousands of nodes. As data volume increases, new RegionServers can be added seamlessly, allowing the system to expand without service interruption.

5. **Fault Tolerance**

HBase ensures reliability through HDFS, where all metadata and data files are stored redundantly.

In case of RegionServer failure, HBase automatically reassigns regions to ensure continuous availability and fault-tolerant operation.

HBase Concepts

A. HBase Data Model

1. Table

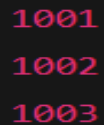
In HBase, a table is a collection of rows, similar to a relational table, but with a flexible and schema-less structure. Only column families must be defined in advance; columns can expand dynamically.

Example: A table named *userinfo*.

2. Row Key

A row key uniquely identifies each row in the table. Rows are stored alphabetically sorted by their row key, which directly influences read performance.

Example: Row keys in *userinfo* table:



1001
1002
1003

3. Column Family

A column family is a logical grouping of related columns, and must be specified at the time of table creation. All columns inside a family are stored together on disk, improving read efficiency.

Example column families for table *userinfo*:

- profile
- activity

So columns will look like:

- profile: name
- profile: email
- activity: last_login
- activity: posts

HBase Concepts continued...

4. Columns (Column Qualifiers)

Columns belong to a column family and can be added dynamically, making HBase highly flexible.

Example: Within profile family:

- profile: name
- profile: email
- profile: phone

These columns do not need to be predefined.

5. Cell

A cell is the intersection of Row key, Column family, Column qualifier, Timestamp where each cell stores a single value.

Example:

```
Row key: 1001
Column: profile:name
Timestamp: 1735720000
Value: "Dipak Gautam"
```

6. Timestamp

HBase maintains multiple versions of the same data using timestamps. The latest timestamp represents the most recent value.

Example (multiple versions of email):

```
profile:email (t=1735720000) → "user123@example.com"
profile:email (t=1735600000) → "user123_old@example.com"
```

HBase Concepts continued...

B. HBase Architecture

HBase follows a distributed and scalable architecture designed to provide real-time access to large datasets. Its architecture is built on top of HDFS and uses a master–slave model supported by a coordination service (ZooKeeper). The main components are the HMaster, RegionServers, Regions, and ZooKeeper.

HBase Components

1. HMaster

The HMaster acts as the central coordinator of the HBase cluster. It is responsible for managing table metadata, assigning regions to RegionServers, performing load balancing, and monitoring cluster health. It does not handle read/write operations directly; instead, it oversees the system's administrative functions.

2. RegionServer

RegionServers are worker nodes that manage and serve data. Each RegionServer hosts multiple regions and is responsible for executing all client read and write operations for those regions. RegionServers also handle region splitting when regions grow large.

Example:

```
Region 1 → row keys: product0001-product0500  
Region 2 → row keys: product0501-product1000
```

3. Regions

A Region is a horizontal partition of an HBase table, containing a contiguous range of row keys. Regions enable parallel processing and allow the system to scale horizontally. As data size increases, regions automatically split to maintain performance and distribute workload.

4. ZooKeeper

ZooKeeper provides the coordination layer for HBase. It tracks active RegionServers, manages master election, and performs failure detection. ZooKeeper ensures that the distributed components of HBase operate reliably and remain synchronized.

HBase Clients

HBase provides multiple client interfaces through which users and applications can interact with the database. These interfaces support administrative operations, data manipulation, and integration with external systems.

A. HBase Shell (Interactive Client)

The HBase Shell is a command-line, interactive interface used primarily for administration, testing, and basic data operations. It supports commands for creating tables, inserting data, retrieving values, and managing schemas.

Common Shell Commands

```
create 'students', 'info'           → creates a table
put 'students', '1', 'info:name', 'Aarav' → inserts data
get 'students', '1'                 → retrieves a row
scan 'students'                     → scans entire table
disable 'students'                  → disables the table
drop 'students'                     → deletes the table
```

B. Java API (Primary Client Interface)

The Java API is the most widely used and fully-featured client interface for HBase. It enables programmatic access to data through operations such as Get, Put, Scan, and Delete. This API is preferred for production applications due to its performance and full feature support.

HBase Clients

C. REST API

The REST API allows clients to access HBase using HTTP requests, making it suitable for web-based systems and platforms that rely on RESTful communication. Data is exchanged using XML, JSON.

D. Thrift API

The Thrift API provides a language-independent interface, enabling applications written in Python, PHP, Ruby, and other non-Java languages to interact with HBase. It uses Apache Thrift to define cross-language service protocols.

E. MapReduce Integration

HBase integrates with the Hadoop MapReduce framework through **HBaseInputFormat** and **HBaseOutputFormat**, enabling large-scale batch processing.

- **HBaseInputFormat** reads rows from HBase as input to MapReduce jobs.
- **HBaseOutputFormat** writes processed results back into HBase tables.

HBase Example

Goal: Store and retrieve student records

Step 1: Create Table

```
create 'students', 'info'
```

Step 2: Insert Data

```
put 'students', '1001', 'info:name', 'John'  
put 'students', '1001', 'info:dept', 'CS'  
put 'students', '1002', 'info:name', 'Asha'
```

Step 3: Read Data

- Get specific row:

```
get 'students', '1001'
```

- Scan entire table

```
scan 'students'
```

HBase Example

Step 4: Update Data

```
put 'students', '1001', 'info:dept', 'IT'
```

Step 5: Delete Data

```
delete 'students', '1002', 'info:name'
```

Step 6: Disable and Drop Table

```
disable 'students'  
drop 'students'
```


Hadoop EcoSystem(HBASE)

- **HBase vs. RDBMS**

Feature	HBase	RDBMS
Data Model	NoSQL, column-oriented	Structured, row-oriented
Schema	Flexible schema (dynamic columns)	Fixed schema (predefined tables and columns)
Scalability	Horizontally scalable (add more nodes)	Vertically scalable (more powerful hardware)
Data Storage	Stores data as key-value pairs in HDFS	Stores data in tables with rows and columns
Query Language	Uses APIs for data access	SQL (Structured Query Language)
Transactions	Limited support for ACID transactions	Full ACID compliance
Performance	Optimized for large datasets and real-time access	Good for complex queries and transactions
Use Cases	Big data applications, real-time data	Traditional applications, transactional systems
Joins	Limited support for joins	Strong support for complex joins
Indexing	Supports secondary indexing	Multi-indexing capabilities

Hadoop EcoSystem(Spark)

Apache Spark is a fast, general-purpose, in-memory data processing engine designed for large-scale data analytics. Unlike Hadoop MapReduce, which uses disk-based computation, Spark performs most operations in memory, significantly improving processing speed.

Spark supports multiple workloads, including batch processing, streaming, machine learning, and graph analytics all within a single unified framework.

Characteristics of Apache Spark

1. In-Memory Processing

Spark keeps most intermediate data in memory (RAM), reducing disk I/O and improving speed by up to 100 times in certain workloads.

2. Distributed Framework

Apache Spark processes data across a cluster of machines, dividing datasets into partitions. Tasks are executed in parallel on executors, enabling scalability, high performance, and fault tolerance.

3. Unified Engine

It supports various types of workloads within a single framework:

- Batch processing
- Real-time stream processing
- Machine learning
- Graph analytics
- Interactive SQL queries

Hadoop EcoSystem(Spark)

4. Language Flexibility

Spark provides APIs in Java, Python, R, SQL which makes it accessible to a wide range of developers and analysts.

5. Compatibility

Spark integrates with many existing Big Data tools such as:

- Hadoop HDFS
- Hive
- HBase
- Cassandra
- Amazon S3

Spark Components

1. Spark Core

Spark Core forms the foundation of the Spark framework. It provides essential functionalities such as task scheduling, memory management, and fault tolerance using Resilient Distributed Datasets (RDDs). Spark Core is responsible for executing distributed jobs, coordinating tasks across cluster nodes, and supporting input/output operations from various storage systems like HDFS, S3, and local file systems. All other Spark components are built on top of Spark Core.

2. Spark SQL

Spark SQL is the module designed for processing structured and semi-structured data. It allows users to perform SQL queries, interact with Hive tables, and work with DataFrames and Datasets. The Catalyst optimizer in Spark SQL optimizes query execution for high performance. Spark SQL supports multiple data sources, including JSON, Parquet, Avro, and JDBC, making it ideal for relational-style analytics on big data.

3. Spark Streaming

Spark Streaming enables real-time data processing by dividing incoming data streams into small micro-batches. It supports integration with sources such as Kafka, Flume, TCP sockets, and Twitter. Spark Streaming provides window operations for time-based computations and ensures fault tolerance through checkpointing and RDD lineage. It is widely used for applications requiring near real-time analytics, such as monitoring and fraud detection.

Spark Components

4. MLlib

MLlib is Spark's scalable machine learning library. It provides a rich set of algorithms for classification, regression, clustering, and collaborative filtering. MLlib also supports feature extraction, transformation, and selection, as well as machine learning pipelines for workflow automation. Built on Spark Core, MLlib leverages distributed computing for efficient processing of large datasets, making it suitable for advanced predictive analytics.

5. GraphX

GraphX is Spark's graph processing module. It represents graphs as vertices and edges and provides pre-built graph algorithms like PageRank, Connected Components, and Shortest Path. GraphX integrates with RDDs, enabling distributed and fault-tolerant computations on large-scale graph data. It is used in applications such as social network analysis, recommendation systems, and network topology computations.

Hadoop EcoSystem(Spark)

RDD (Resilient Distributed Dataset)

A Resilient Distributed Dataset (RDD) is the core abstraction and fundamental data structure of Apache Spark. It represents an immutable, distributed collection of objects that can be processed in parallel across a cluster. RDDs provide the foundation for all Spark operations, enabling efficient, fault-tolerant, and scalable computation on large datasets.

Key Features:

1. Immutable:

Once created, an RDD cannot be altered. Any transformation applied to an RDD results in the creation of a new RDD, ensuring data consistency and supporting reliable computations in a distributed environment.

2. Distributed:

RDDs are automatically partitioned across multiple cluster nodes, enabling parallel execution of tasks. This distribution allows Spark to handle very large datasets efficiently.

3. Fault Tolerance:

RDDs maintain lineage information, which records the sequence of transformations used to create them. In case of node failure or data loss, lost partitions can be recomputed from the lineage, ensuring resiliency.

Hadoop EcoSystem(Spark)

4. **In-Memory Computation:**

RDDs can be cached or persisted in memory, which significantly reduces the need to read from disk repeatedly. This improves processing speed for iterative and interactive algorithms.

5. **Lazy Evaluation:**

Transformations on RDDs are not executed immediately. Spark builds a Directed Acyclic Graph (DAG) of operations and executes them only when an action (such as `collect()` or `count()`) is called. This allows optimization of computation and reduces unnecessary data movement.

6. **Rich Operations:**

RDDs support two types of operations:

- **Transformations:** Create new RDDs from existing ones (e.g., `map()`, `filter()`, `flatMap()`).
- **Actions:** Trigger execution and return results (e.g., `collect()`, `count()`, `reduce()`).

Thank You!